

Safey A.Halim

halims@in.tum.de

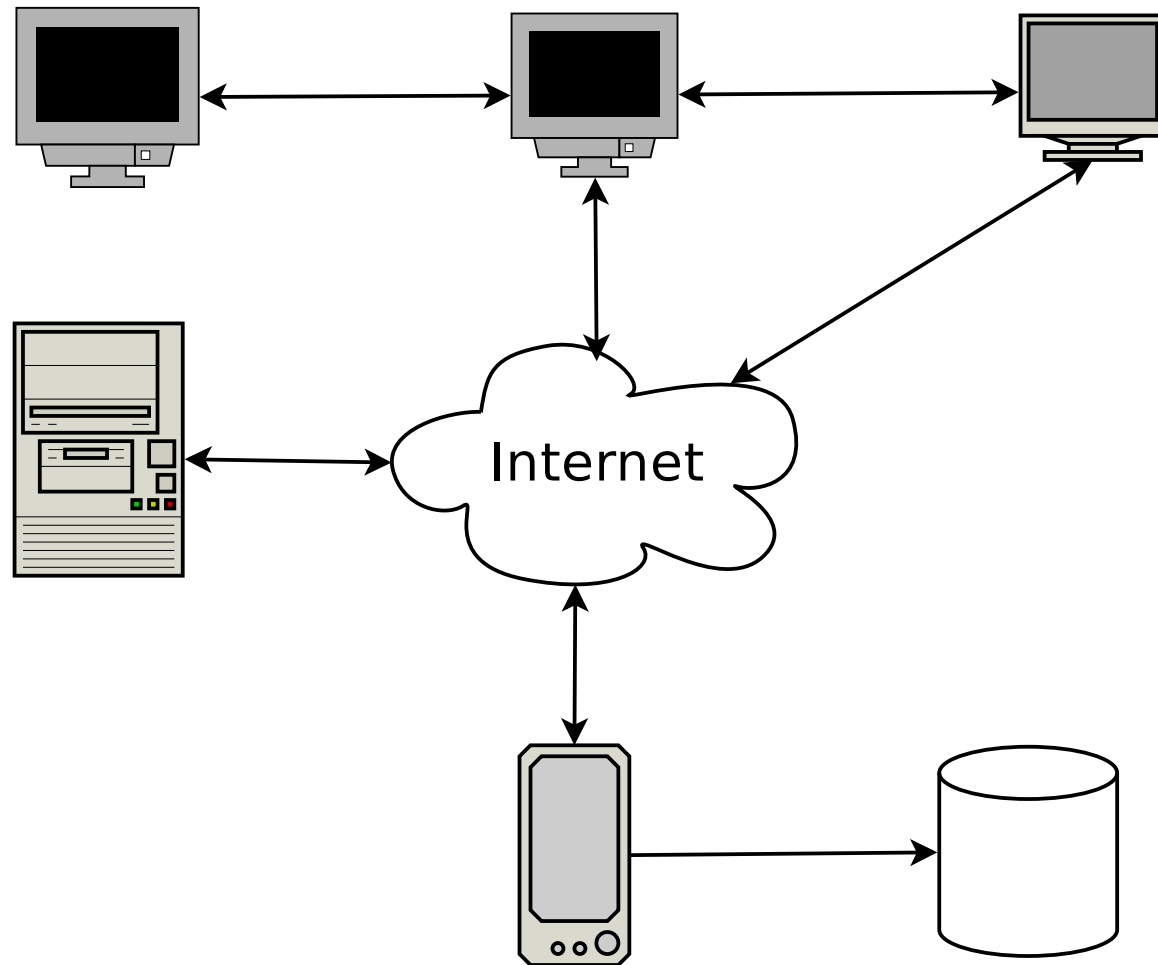
Monkey: Automated Debugging of Deployed  
Distributed Systems

Advisor: Christian Grothoff

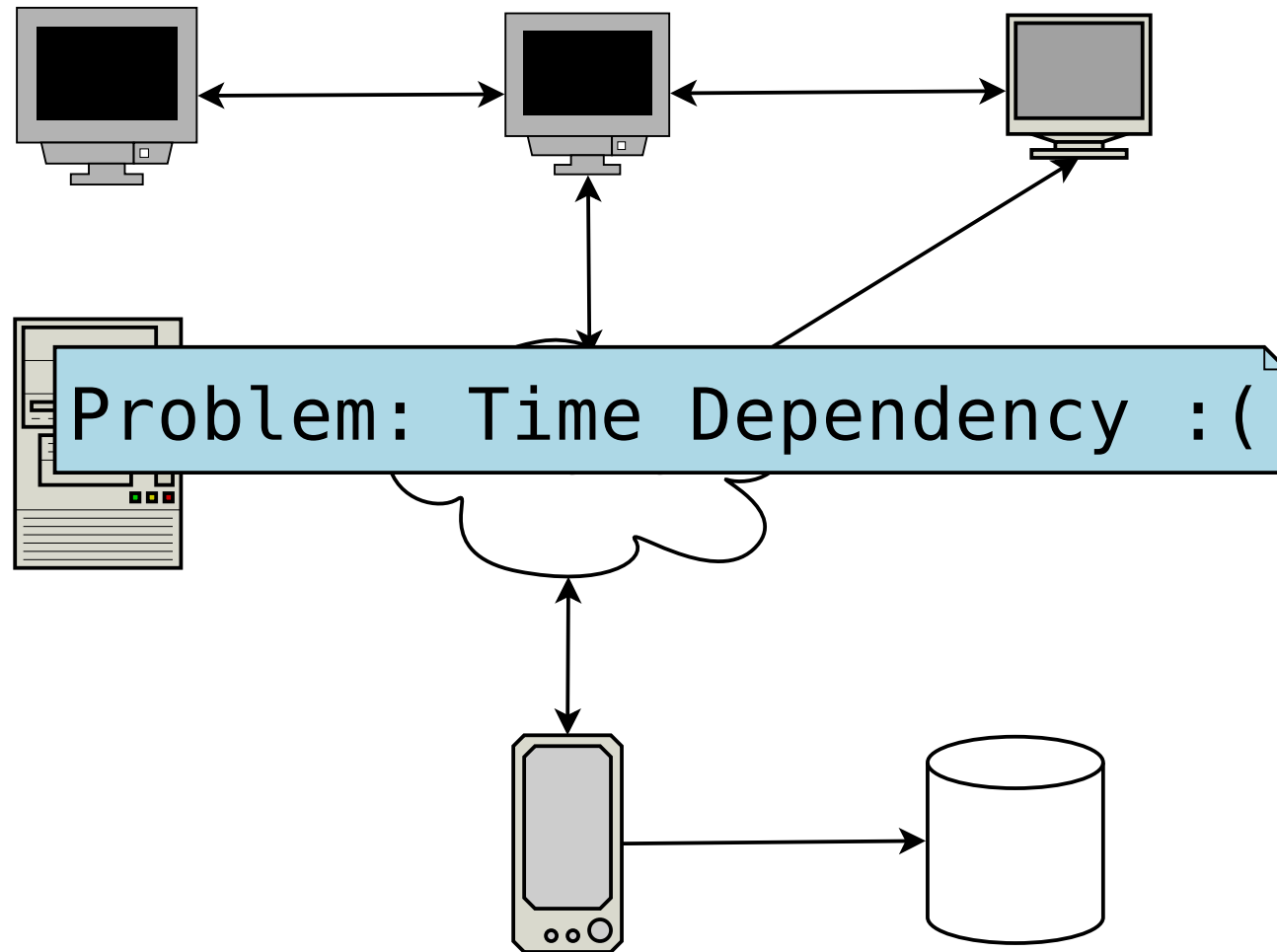




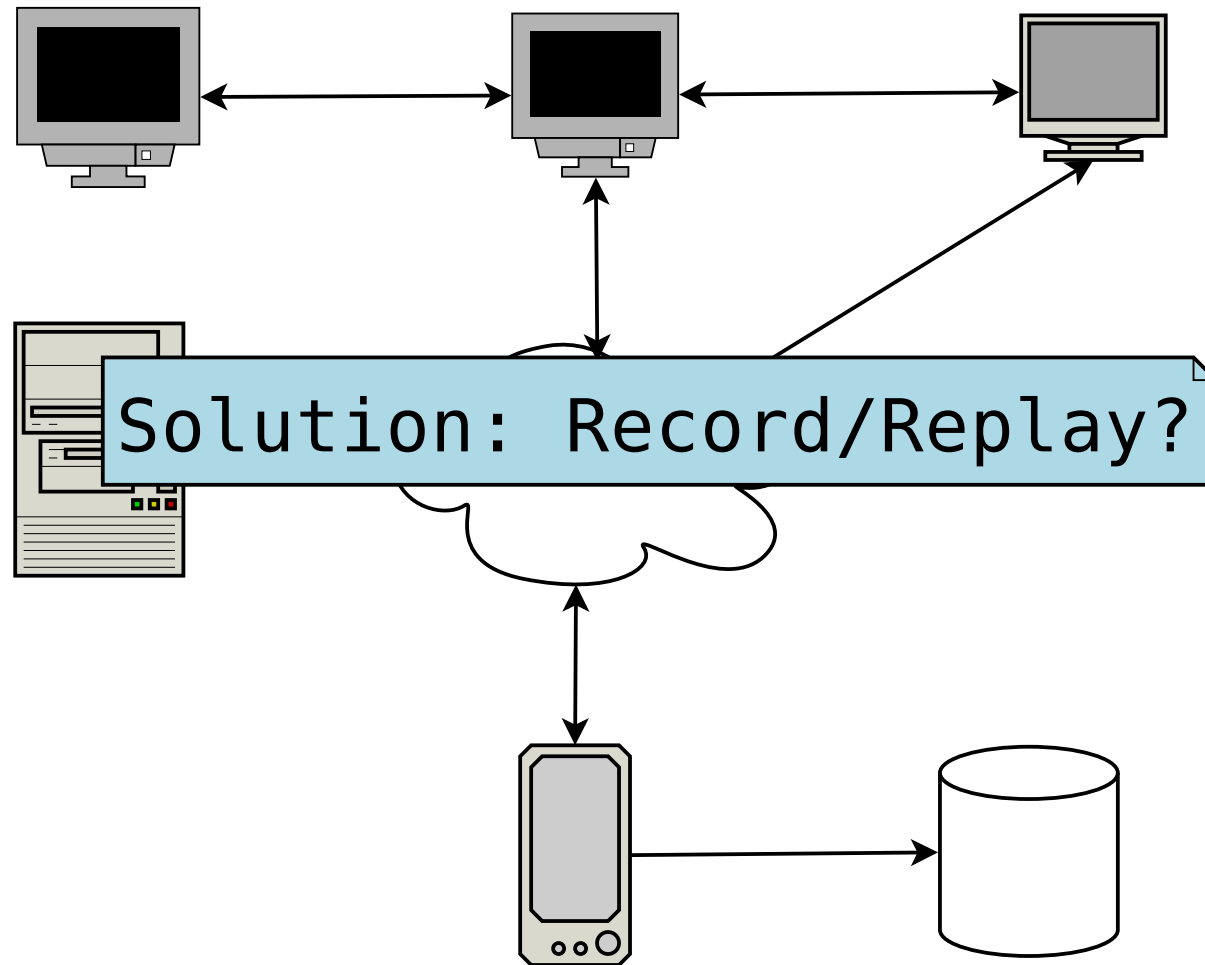
# Distributed Systems



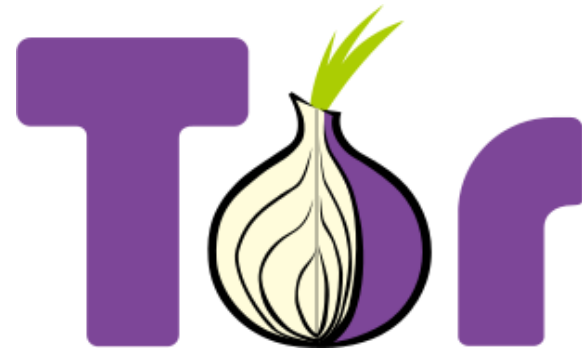
# Distributed Systems



# Distributed Systems



# Distributed Systems

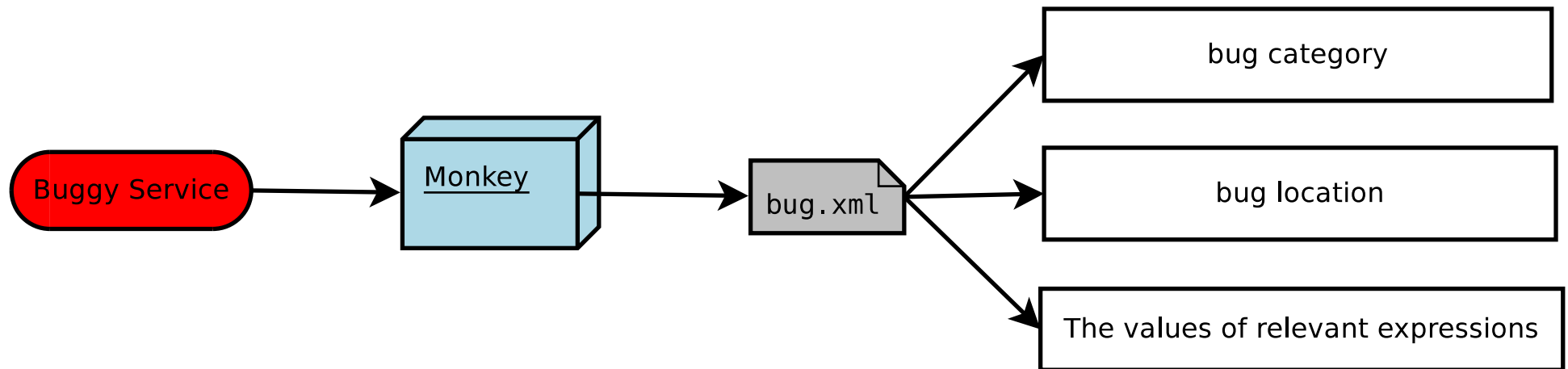


# Proposed Solution

- Autonomous
- Real time
- No whole system recording
- Monkey?!



# Monkey: Idea





# Monkey: Idea-Example

```
#include <stdio.h>
#include <assert.h>

void assertionFailure()
{
    int x;
    x = 5;
    printf("Assertion Failure Now!\n");
    assert(x < 4);
}

int main(int argc, char *argv[])
{
    assertionFailure();
    return 0;
}
```



# Monkey: Idea-Example

```

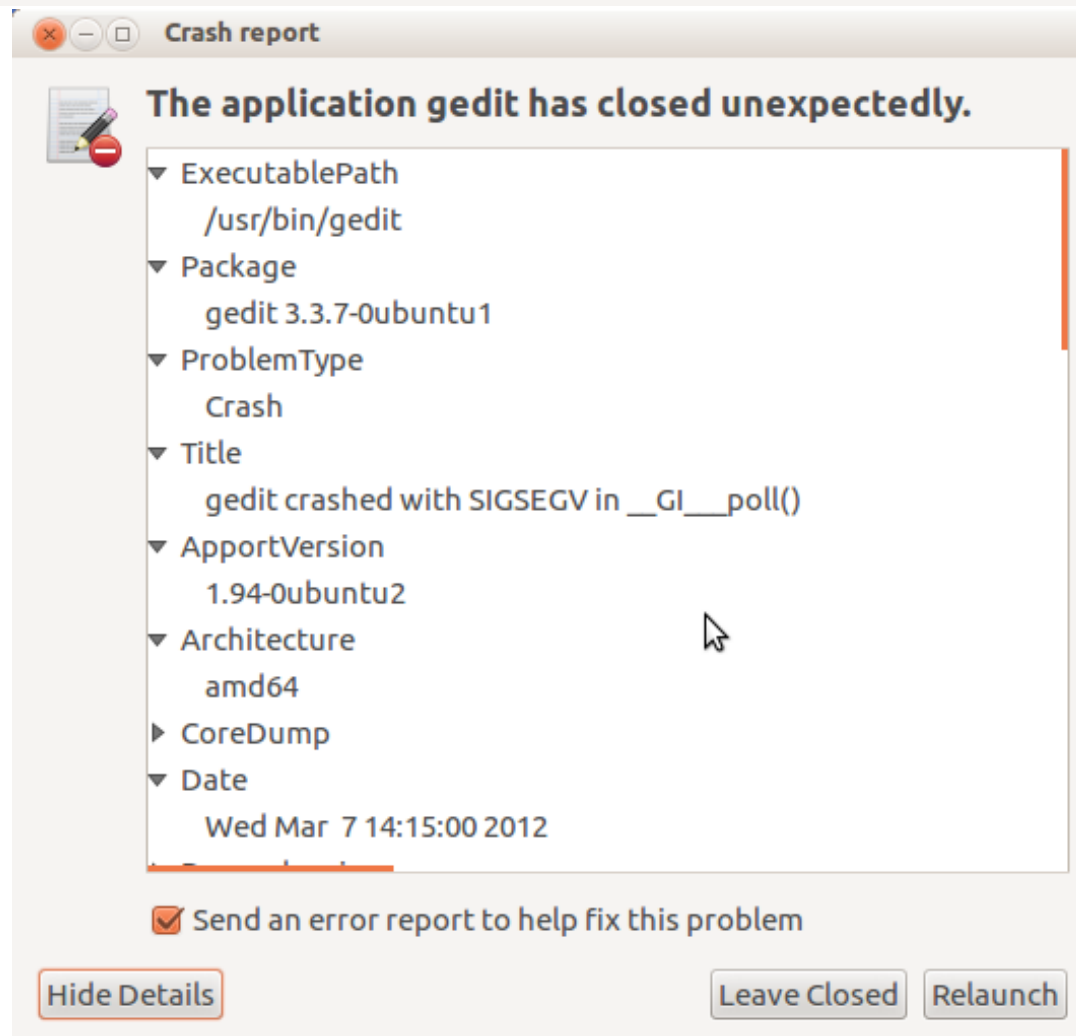
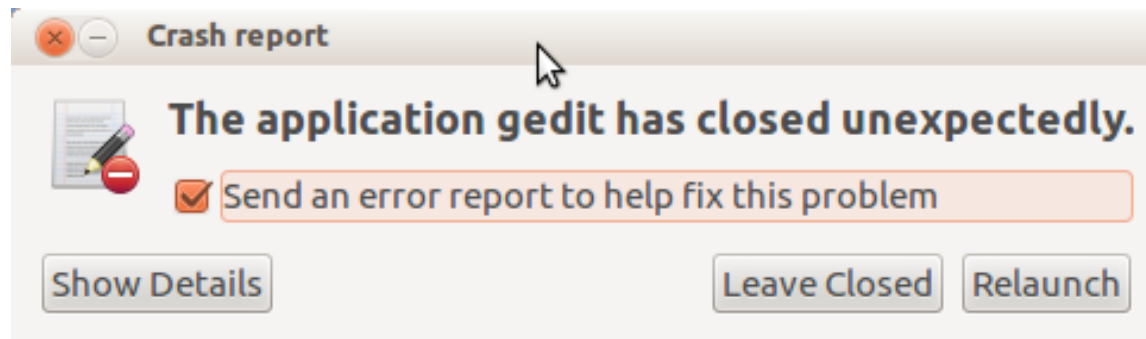
▼<crash category="Assertion Failure" function="assertionFailure" line="9" file="bug_assertion_failure.c">
  ▼<history>
    ▼<epoch step="0">
      ▼<trace>
        ▼<function name="assertionFailure" line="9" file="bug_assertion_failure.c" depth="0">
          ▼<expressions>
            <expression name="assert(x<4)">Not Evaluated</expression>
            <expression name="x<4">0</expression>
            <expression name="printf("Assertion Failure Now!\n")">Not Evaluated</expression>
            <expression name="x">5</expression>
          </expressions>
        </function>
        ▼<function name="main" line="14" file="bug_assertion_failure.c" depth="1">
          ▼<expressions>
            <expression name="assertionFailure()">Not Evaluated</expression>
            <expression name="argv">0xbffff8c4</expression>
            <expression name="argc">1</expression>
          </expressions>
        </function>
      </trace>
    </epoch>
  </history>
</crash>

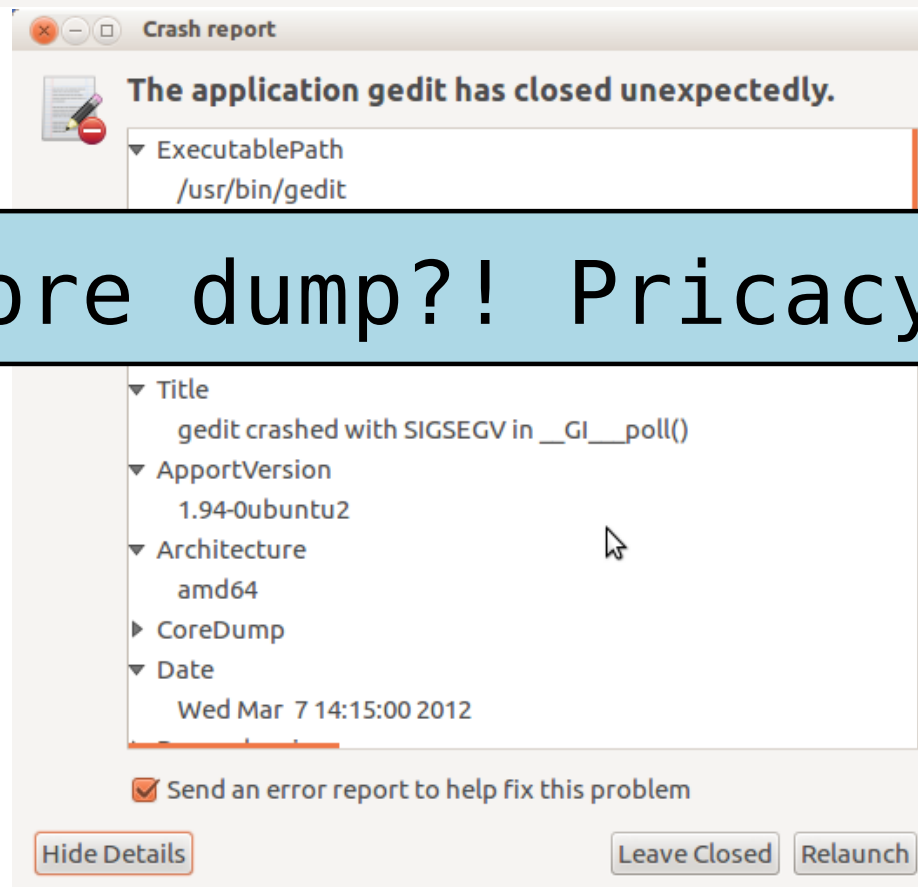
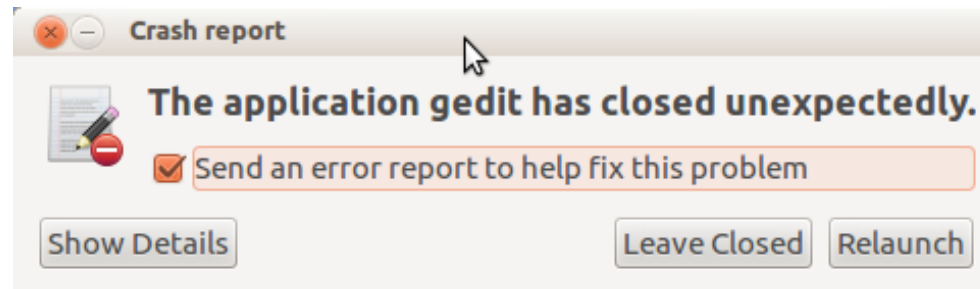
```



# But wait. . .







Core dump?! Privacy?!



# Related problem: Bug Report Redundancy

- Users/Developers submit bug reports
- Chances of duplicates
- Current triaging solutions
  - Bug summary
  - Bug description
- Can we do better? (Monkey?)



# Automated debugging: Background

- Static slicing
  - A set of variables that *\*may\** affect the value of a variable
  - For a program  $P$ ,  $C=(x,V)$
  - Data and control dependencies
- Dynamic slicing
  - A set of variables that *\*actually\** affect the value of a variable
- Postmortem slicing (Monkey)



# Slicing: example

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    int x = 0;
    int y = 5;
    int z = 3;
    if (argc > 5)
        x = y;
    else
        x = z;
    printf("x = %d", x);
    return 0;
}
```

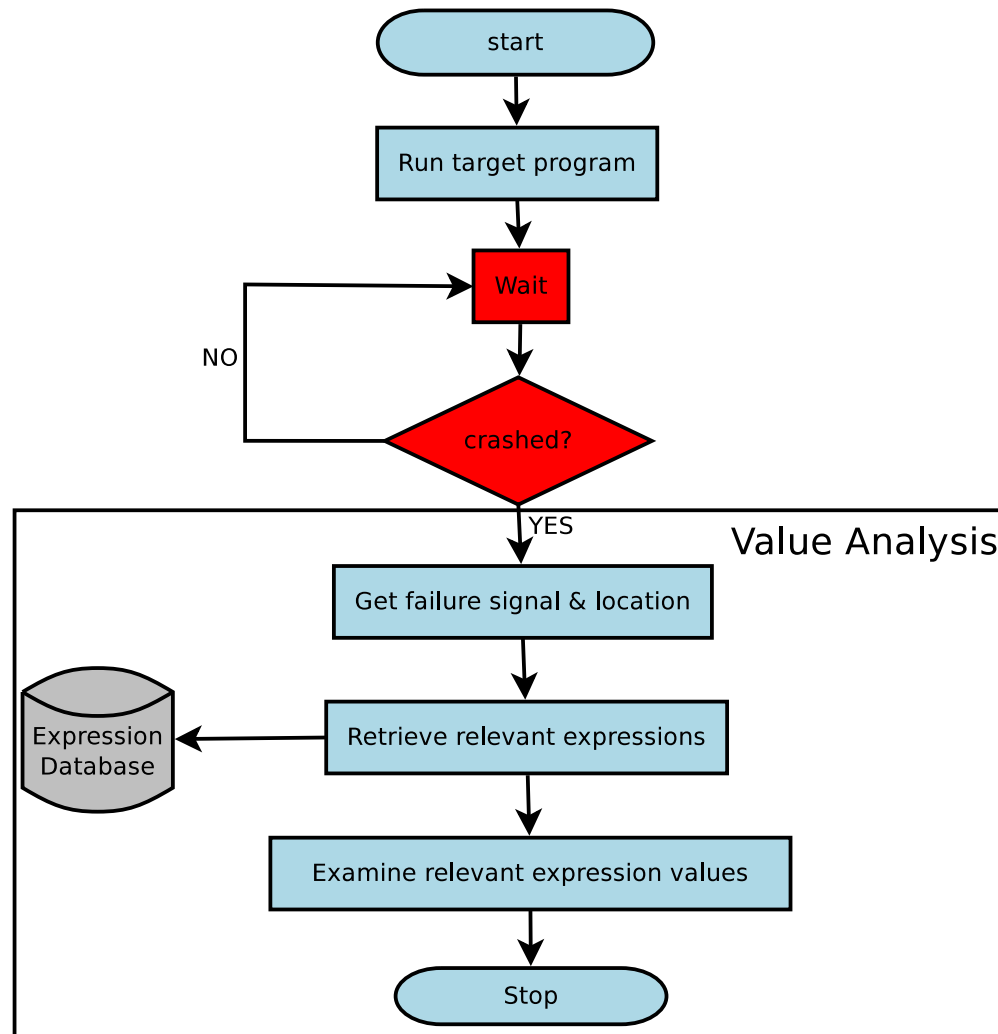
Static slicing: y and z

Dynamic slicing: argc?

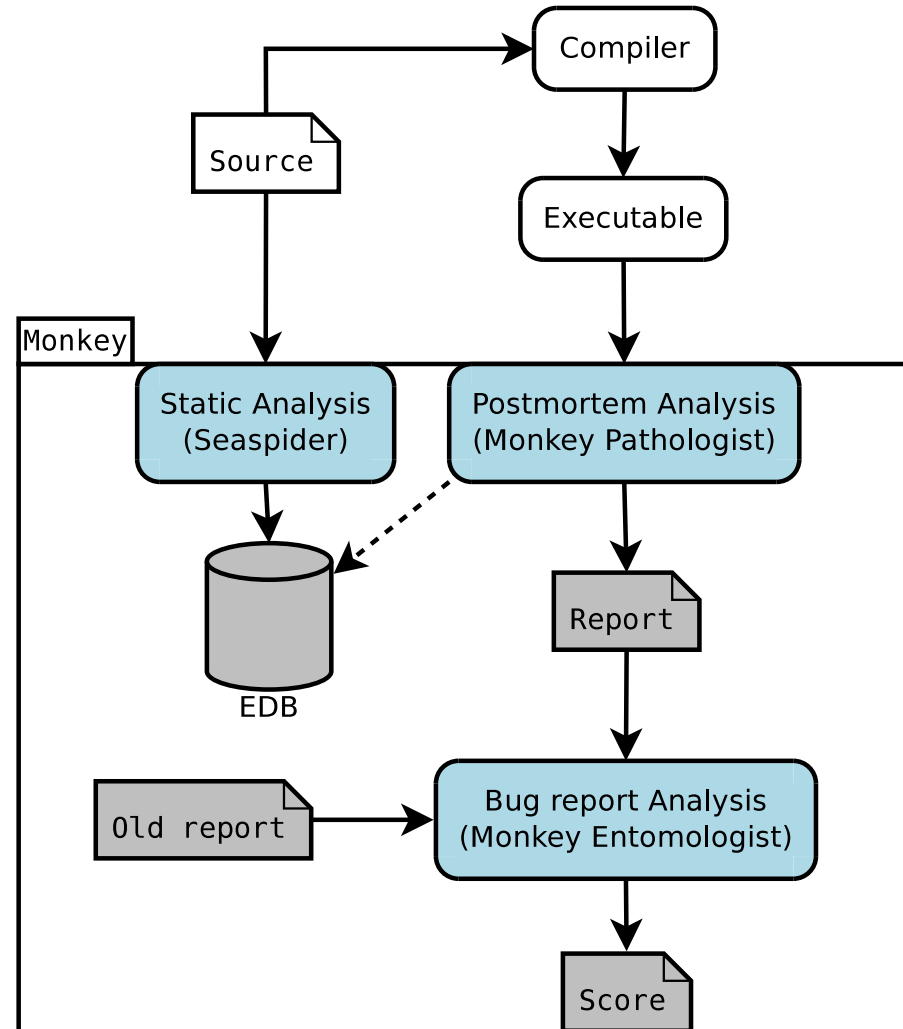




# Slicing: Postmortem slicing

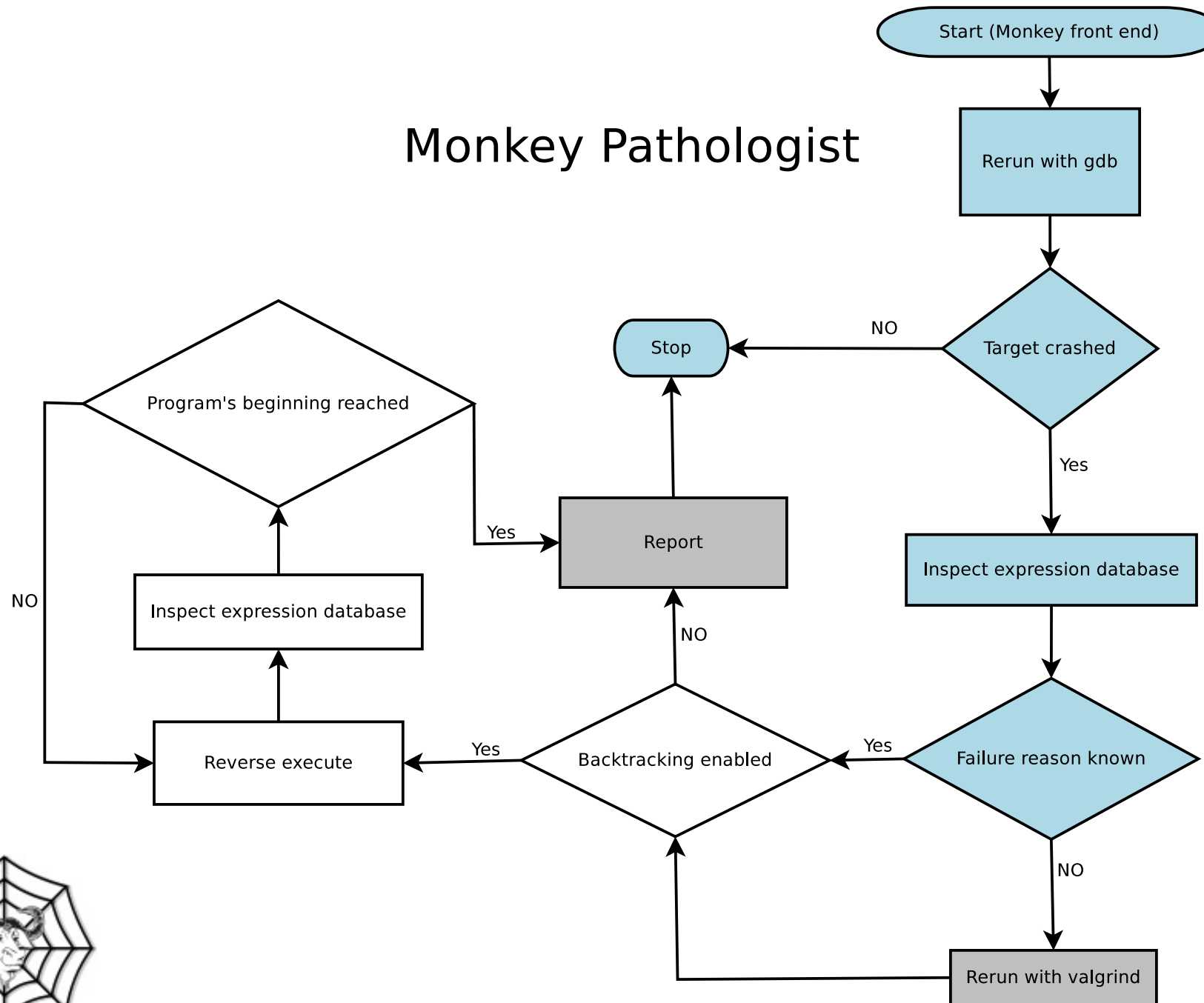


# System Architecture





# Monkey Pathologist

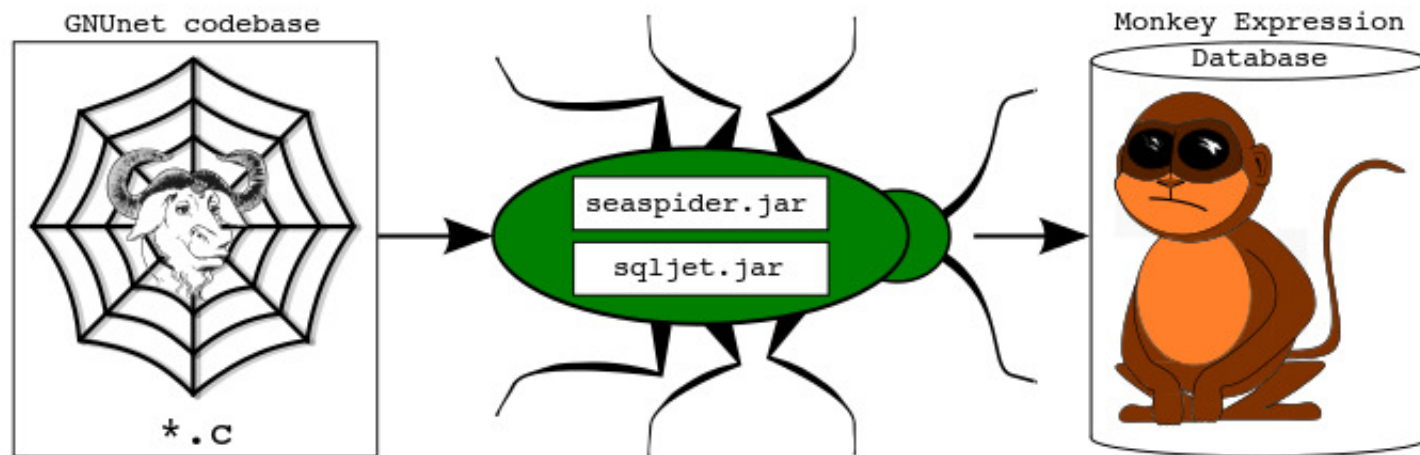


# Pathologist's Features

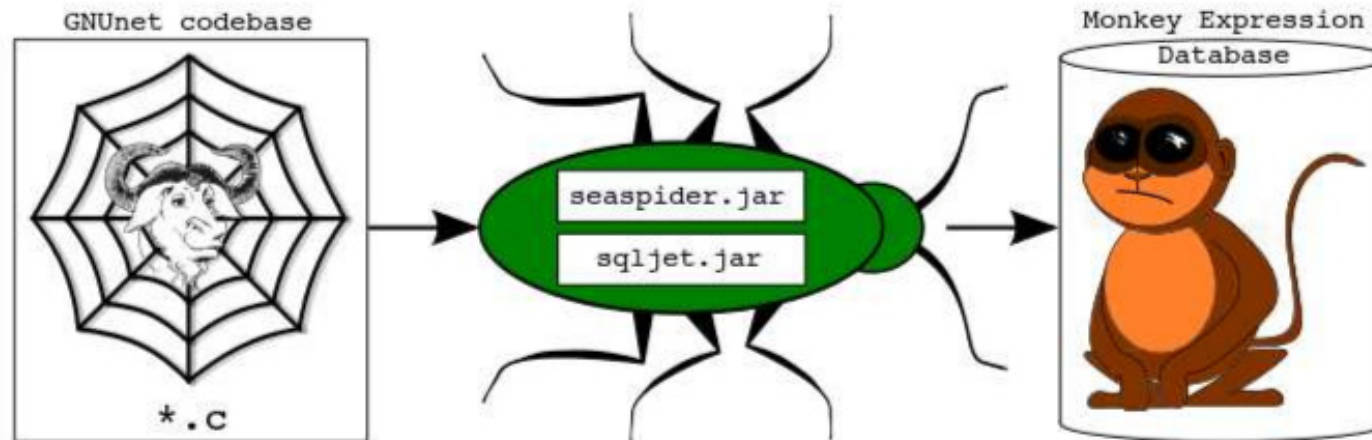
- Debugging C programs
- Characterize the following bugs:
  - Null pointer exception
  - Bad memory access (valgrind)
  - Division by zero
  - Assertion failure



# Seaspider



# Seaspider



# Entomologist

- Input: 2 Monkey bug reports
- Output: Similarity Score
- Based of feature weights:
  - General properties feature: bug category, file name, function name, line number
  - Stack trace feature: function properties, expression properties





# Entomologist contd.

```
<crash category="Assertion Failure" function="assertionFailure" line="9" file="bug_assertion_failure.c">
  <history>
    <epoch step="0">
      <trace>
        <function name="assertionFailure" line="9" file="bug_assertion_failure.c" depth="0">
          <expressions>
            <expression name="assert(x<4)">Not Evaluated</expression>
            <expression name="x<4">0</expression>
            <expression name="printf("Assertion Failure Now!\n")">Not Evaluated</expression>
            <expression name="x">5</expression>
          </expressions>
        </function>
        <function name="main" line="14" file="bug_assertion_failure.c" depth="1">
          <expressions>
            <expression name="assertionFailure">Not Evaluated</expression>
            <expression name="argv">0xbfffe000</expression>
            <expression name="argc">1</expression>
          </expressions>
        </function>
      </trace>
    </epoch>
  </history>
</crash>
```

General Properties

Function Properties

Expression Properties



# Demo



# Future Work

- Presentation:
  - Web Interface: Code annotation
  - Eclipse Plugin
- Seaspider:
  - Inter-procedural analysis (function pointers?)
  - Local flow analysis (an if condition is more important than a printf)



# Future Work contd.

- Pathologist:
  - Manual annotations to enable/disable backtracking
  - Support of other signals
  - Experimenting with: GUNet, Tor, . . .
- Entomologist:
  - Monkey bug report database
  - Fine tuning weights and set thresholds



# *Questions?*



# RTFL

Copyright (C) 2012 Safey A.Halim

Verbatim copying and distribution of this entire article is permitted in any medium, provided this notice is preserved.

